[20] Correctly implement Dijkstra's algorithm and the functionality discussed above. Include a copy of your (well-documented) code in your submission to the TA.
[20] Correctly implement both versions of a priority queue, one using an array with worst case O(1), O(1) and O(|V|) operations and one using a heap with worst case O(log|V|) operations.

```
#!/usr/bin/python3


from CS312Graph import *
import time

class ArrayQueue:
    def __init__(self, dist): #makequeue() function, initializes queue, O(|V|) time, O(|V|) space
        self.arrayList = []
        self.size = 0

        for i in range(len(dist)):
            self.insert(dist[i])
        self.size = len(dist)

    def insert(self, distance): #adds new vertex to queue, O(1) time, O(|V|) space
        self.arrayList.append(distance)
        return

    def deleteMin(self): #removes old vertex from queue, O(|V|) time, O(|V|) space
        min = float('inf')
        remove = 0
        for i in range(len(self.arrayList)):
            if (self.arrayList[i] != None):
                if (self.arrayList[i] < min):
                    min = self.arrayList[i]
                    remove = i
        self.arrayList[remove] = None
        self.size = self.size-1
        if (self.size == 0):
            self.arrayList.clear()
        return min

    def decreaseKey(self, index, distance): #gives vertex new distance, O(1) time, O(|V|) space
        self.arrayList[index] = distance
        return

class HeapQueue:
    def __init__(self, dist): #makequeue() function, initializes queue, O(|V|) time, O(|V|) space
        self.heapList = []
        self.pointerList = []
        self.size = 0

        for i in range(len(dist)):
            self.insert(i, dist[i])

    def insert(self, node, distance): #adds new vertex to queue, O(1) time, O(|V|) space
        self.pointerList.append(len(self.heapList))
        self.heapList.append((node, distance))
        self.size = self.size + 1
        return
```

```python
def deleteMin(self): #removes old vertex from queue, O(log|V|) time, O(|V|) space
    if (len(self.heapList) == 1):
        nodef, distancef = self.heapList.pop()
        self.size = self.size - 1
        return nodef

    nodef, distancef = self.heapList[0]
    self.pointerList[nodef] = None
    self.heapList[0] = self.heapList[-1]
    self.heapList.pop()

    nodel, distancel = self.heapList[0]
    self.pointerList[nodel] = 0
    self.bubbleDown(nodel)

    self.size = self.size-1

    return nodef

def bubbleDown(self, node): #reorganizes queue after delete, O(log|V|) time, O(|V|) space
    while (True):
        nodeloc = self.pointerList[node]
        locc1 = round((nodeloc + 0.5) * 2)
        locc2 = (nodeloc + 1) * 2
        node, nodedist = self.heapList[nodeloc]

        locmax = len(self.heapList) - 1
        if (locc1 > locmax and locc2 > locmax):
            break
        else:
            if (locc1 > locmax):
                distc1 = float('inf')
                idc2, distc2 = self.heapList[locc2]
            elif (locc2 > locmax):
                idc1, distc1 = self.heapList[locc1]
                distc2 = float('inf')
            else:
                idc1, distc1 = self.heapList[locc1]
                idc2, distc2 = self.heapList[locc2]

        if (distc1 <= distc2):
            child = idc1
            childdist = distc1
            childloc = locc1
        else:
            child = idc2
            childdist = distc2
            childloc = locc2

        if (nodedist > childdist):
            self.heapList[childloc] = self.heapList[nodeloc]
            self.pointerList[node] = childloc
            self.heapList[nodeloc] = (child, childdist)
            self.pointerList[child] = nodeloc
        else:
            break
```

```
    def decreaseKey(self, node, distance): #gives vertex new distance, O(log|V|) time, O(|V|) space
        location = self.pointerList[node]
        self.heapList[location] = (node, distance)
        self.bubbleUp(node)
        return

    def bubbleUp(self, node): #reorganizes queue after distance change, O(log|V|) time, O(|V|) space
        while (True):
            nodeloc = self.pointerList[node]
            if (nodeloc == 0):
                break
            locp = (nodeloc - 1) // 2
            node, nodedist = self.heapList[nodeloc]
            idp, distp = self.heapList[locp]
            if (nodedist < distp):
                self.heapList[locp] = self.heapList[nodeloc]
                self.pointerList[node] = locp
                self.heapList[nodeloc] = (idp, distp)
                self.pointerList[idp] = nodeloc
            else:
                break

class NetworkRoutingSolver:
    def __init__( self):
        self.dist = None
        self.prev = None
        self.source = None
        self.dest = None
        self.network = None

    def initializeNetwork( self, network ):
        assert( type(network) == CS312Graph )
        self.network = network

    def getShortestPath( self, destIndex ): #sends path to graph function, O(|V|+|E|) time, O(|V|+|E|) space
        self.dest = destIndex
        path_edges = []
        total_length = 0

        if (len(self.dist) == 0):
            return {'cost': float('inf'), 'path': path_edges}
        else:
            total_length = self.dist[self.dest]

            currIndex = self.dest
            prevIndex = None
            while (currIndex != self.source):
                prevIndex = self.prev[currIndex]
                currNode = self.network.nodes[currIndex]
                if (prevIndex == None):
                    return {'cost': float('inf'), 'path': path_edges}
                prevNode = self.network.nodes[prevIndex]
                for i in range(len(prevNode.neighbors)):
                    if (prevNode.neighbors[i].dest == currNode):
                        edge = prevNode.neighbors[i]
```

```
                break
            path_edges.append( (currNode.loc, prevNode.loc, '{:.0f}'.format(edge.length)) )
            currIndex = prevIndex

    return {'cost':total_length, 'path':path_edges}

def computeShortestPaths( self, srcIndex, use_heap=False ): #finds shortest path via Dijkstra's, O(see below)
    self.source = srcIndex
    t1 = time.time()
    self.dist = []
    self.prev = []

    for i in range(len(self.network.nodes)):
        self.dist.append(float('inf'))
        self.prev.append(None)
    self.dist[self.source] = 0

    if (use_heap == False): #array implementation, O(|V|^2) + O(|E|) time, O(|V|+|E|) space
        arrayqueue = ArrayQueue(self.dist);
        while (arrayqueue.size > 0):
            min = arrayqueue.deleteMin()
            if (min < float('inf')):
                i = self.dist.index(min)
                for j in range(len(self.network.nodes[i].neighbors)):
                    k = self.network.nodes.index(self.network.nodes[i].neighbors[j].dest)
                    if (k >= 0):
                        n = self.dist[i] + self.network.nodes[i].neighbors[j].length
                        if (self.dist[k] > n):
                            self.dist[k] = n
                            self.prev[k] = i
                            arrayqueue.decreaseKey(k, n)
            else:
                break
    else: #heap implementation, O(|V|log|V|) + O(|E|log|V|) time, O(|V|+|E|) space
        heapqueue = HeapQueue(self.dist);
        heapqueue.decreaseKey(srcIndex, 0)
        while (heapqueue.size > 0):
            min = heapqueue.deleteMin()
            if (self.dist[min] != float('inf')):
                for j in self.network.nodes[min].neighbors:
                    k = j.dest.node_id
                    n = self.dist[min] + j.length
                    if (self.dist[k] == float('inf') or n < self.dist[k]):
                        self.dist[k] = n
                        self.prev[k] = min
                        heapqueue.decreaseKey(k, n)
            else:
                break

    t2 = time.time()
    return (t2-t1)
```

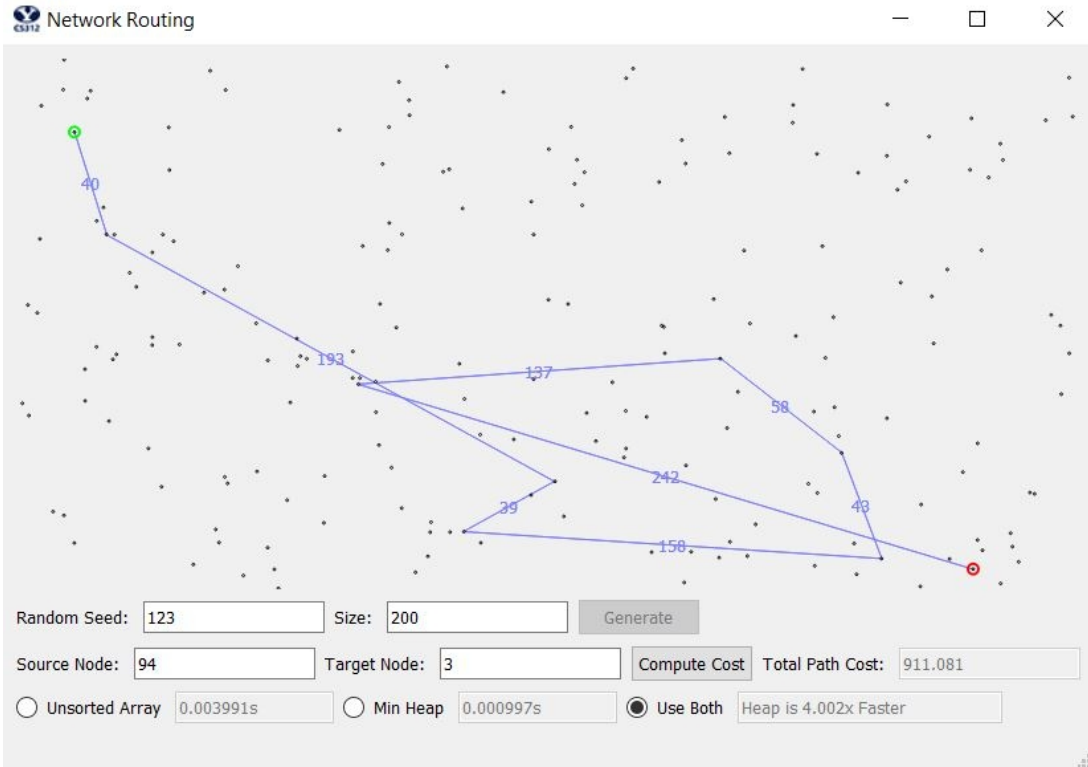
For each operation (insert, delete-min, and decrease-key) convince us (refer to your included code) that the complexity is what is required here.

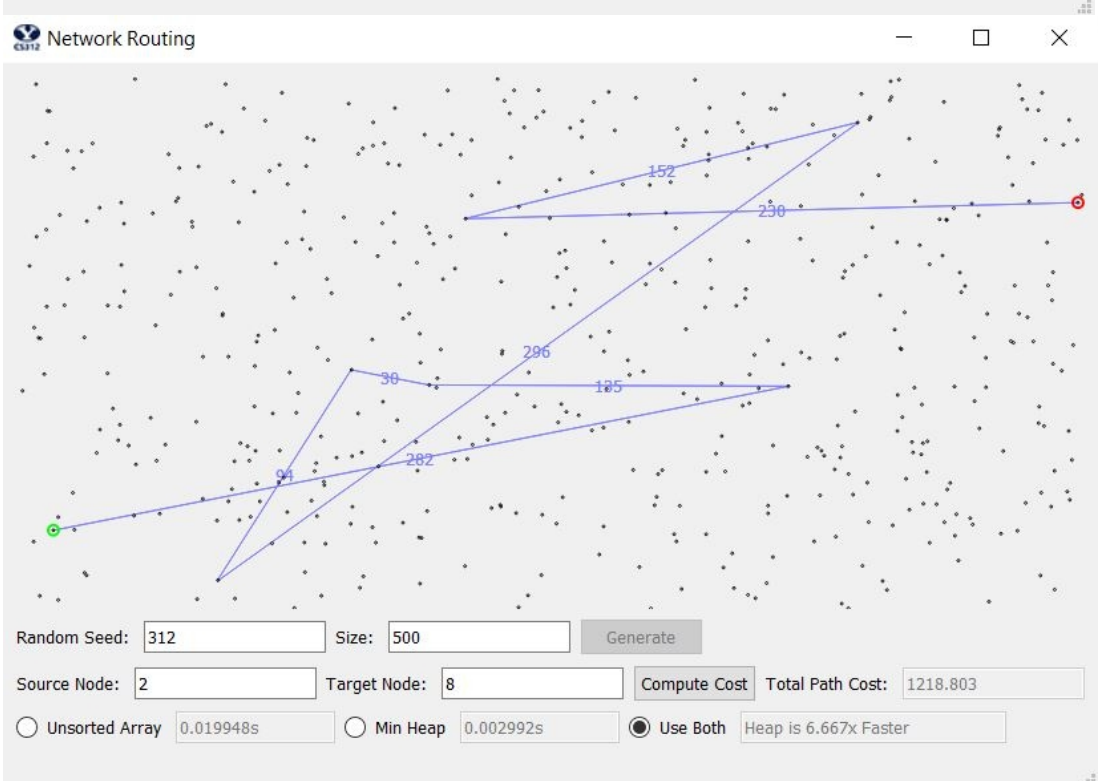
| | insert() | delete-min() | decrease-key() |
|---|---|---|---|
| Array implementation: | Contains one command to append a new value, making it O(1). | Contains one for loop that loops through a max of all vertices, making it O(\|V\|). | Contains one command to change a specific value, making it O(1). |
| Heap implementation: | Contains three constant commands, making it O(1). | Contains bubble-down, making it O(log\|V\|). | Contains bubble-up, making it O(log\|V\|). |

[10 points] Explain the time and space complexity of both implementations of the algorithm by showing and summing up the complexity of each subsection of your code.

| | Array implementation: | Heap implementation: |
|---|---|---|
| makequeue() | O(\|V\|) for queue array = O(\|V\|) time.<br>O(\|V\|) for queue array = O(\|V\|) space. | O(\|V\|) for heap+pointer arrays = O(\|V\|) time.<br>O(\|V\|) for heap+pointer arrays = O(\|V\|) space. |
| insert() | O(1) time.<br>O(\|V\|) for queue array = O(\|V\|) space. | O(1) time.<br>O(\|V\|) for heap+pointer arrays = O(\|V\|) space. |
| delete-min() | O(\|V\|) time.<br>O(\|V\|) for queue array = O(\|V\|) space. | O(log\|V\|) time.<br>O(\|V\|) for heap+pointer arrays = O(\|V\|) space. |
| decrease-key() | O(1) time.<br>O(\|V\|) for queue array = O(\|V\|) space. | O(log\|V\|) time.<br>O(\|V\|) for heap+pointer arrays = O(\|V\|) space. |
| Dijkstra's | O(\|V\|*deletemin) + O(\|E\|*decreasekey) = O($\|V\|^2$) + O(\|E\|) time.<br>O(\|V\|+\|E\|) for queue+edges = O(\|V\|+\|E\|) space. | O(\|V\|*deletemin) + O(\|E\|*decreasekey) = O(\|V\|log\|V\|) + O(\|E\|log\|V\|) time.<br>O(\|V\|+\|E\|) for queue+edges = O(\|V\|+\|E\|) space. |

[20] For Random seed 42 - Size 20, Random Seed 123 - Size 200 and Random Seed 312 - Size 500, submit a screenshot showing the shortest path (if one exists) for each of the three source-destination pairs, as shown in the images below.
For Random seed 42 - Size 20, use node 7 (the left-most node) as the source and node 1 (on the bottom toward the right) as the destination, as in the first image below.
For Random seed 123 - Size 200, use node 94 (near the upper left) as the source and node 3 (near the lower right) as the destination, as in the second image below.
For Random seed 312 - Size 500, use node 2 (near the lower left) as the source and node 8 (near the upper right) as the destination, as in the third image below.


Network Routing
40
193
137
58
242
39
43
158
Random Seed: 123
Size: 200
Generate
Source Node: 94
Target Node: 3
Compute Cost
Total Path Cost: 911.081
Unsorted Array
0.003991s
Min Heap
0.000997s
Use Both
Heap is 4.002x Faster


Network Routing
152
230
296
30
185
282
94
Random Seed: 312
Size: 500
Generate
Source Node: 2
Target Node: 8
Compute Cost
Total Path Cost: 1218.803
Unsorted Array
0.019948s
Min Heap
0.002992s
Use Both
Heap is 6.667x Faster

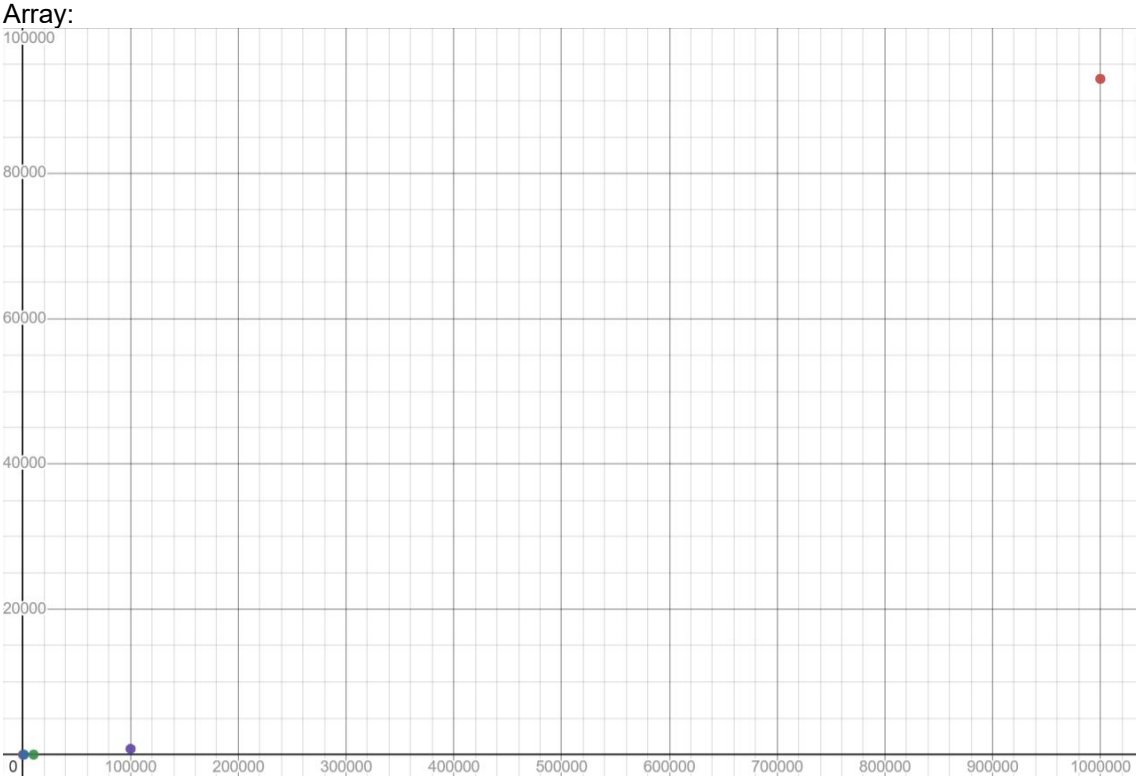
[20] For different numbers of nodes (100, 1000, 10000, 100000, 1000000), compare the empirical time complexity for Array vs. Heap, and give your best estimate of the difference (for 1000000 nodes, run only the heap version and then estimate how long you might expect your array version to run based on your other results). For each number of nodes do at least 5 tests with different random seeds, and average the results. Redo any case where the destination is unreachable. Each time, start with nodes approximately in opposite corners of the network. Graph your results and also give a table of your raw data (data for each of the runs); in both graph and table, include your one estimated runtime (array implementation for 1000000 points).

| Array: | | | | | | |
|---|---|---|---|---|---|---|
| Points | Test #1 | Test #2 | Test #3 | Test #4 | Test #5 | Average |
| 100 | 0.001 | 0.001 | 0.001 | 0.001 | 0.001 | 0.001 |
| 1000 | 0.079 | 0.078 | 0.079 | 0.079 | 0.076 | 0.078 |
| 10000 | 7.942 | 7.976 | 7.902 | 7.908 | 7.859 | 7.917 |
| 100000 | 925 | 955 | 863 | 961 | 948 | 930 |
| 1000000 | 93000 | N/A | N/A | N/A | N/A | 93000 |

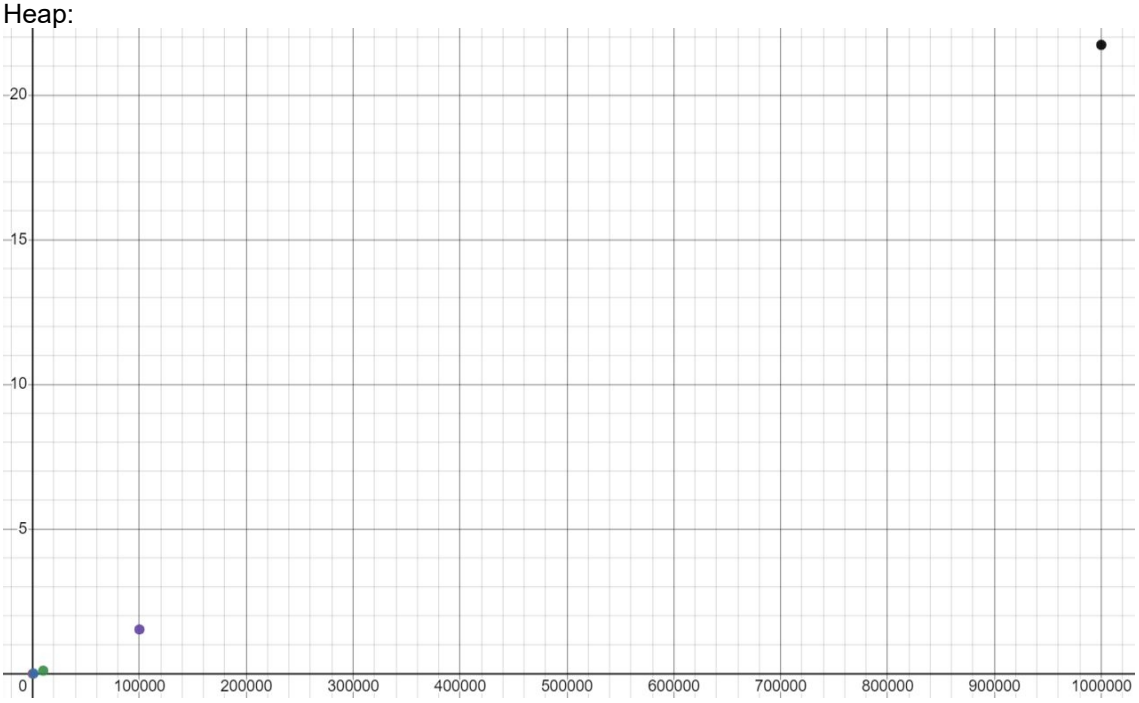
| Heap: | | | | | | |
|---|---|---|---|---|---|---|
| Points | Test #1 | Test #2 | Test #3 | Test #4 | Test #5 | Average |
| 100 | 0.001 | 0.001 | 0.001 | 0.001 | 0.001 | 0.001 |
| 1000 | 0.007 | 0.007 | 0.007 | 0.007 | 0.007 | 0.007 |
| 10000 | 0.099 | 0.103 | 0.122 | 0.102 | 0.103 | 0.106 |
| 100000 | 1.527 | 1.509 | 1.526 | 1.539 | 1.553 | 1.531 |
| 1000000 | 20.830 | 21.846 | 21.157 | 22.008 | 22.906 | 21.749 |

| Faster: | | | | | | |
|---|---|---|---|---|---|---|
| Points | Test #1 | Test #2 | Test #3 | Test #4 | Test #5 | Average |
| 100 | 1.033x | 1.041x | 0.934x | 0.989x | 0.980x | 0.995x |
| 1000 | 11.268x | 11.146x | 11.333x | 11.433x | 11.000x | 11.236x |
| 10000 | 80.438x | 77.653x | 64.942x | 77.735x | 76.512x | 75.456x |
| 100000 | 605.763x | 632.896x | 565.531x | 634.431x | 610.431x | 609.810x |
| 1000000 | 4464.714x | N/A | N/A | N/A | N/A | 4276.059x |

Array:




Heap:




Discuss the results and give your best explanations of why they turned out as they did.
The time complexities for both implementations are polynomial. The heap implementation is significantly faster especially at large values where the |V or E|log|V| term is significantly smaller than the $|V|^2$ term. At smaller values, the speeds are more similar as the algorithm's overhead and the constant factors have a larger impact. The results show $|V|^2$ and |V or E|log|V| growth in run-times which is what we expected from the time complexity explanation.